Diagnóstico

La letra del obligatorio plantea dos problemas:

> **Problema 1 (Conteo incorrecto de batería crítica)**: "El sistema me dice que hay 0 drones con batería crítica, pero yo cuento varios a simple vista en el archivo."
>
> **Problema 2 (Drones sin batería asignados a repartos)**: "Un dron con 5% de batería fue asignado a un reparto largo y se quedó sin energía a mitad de camino. El sistema debería haberlo bloqueado."

Respuesta al problema 1

```
16        tail -n +2 "$ARCHIVO_FLOTA" | while IFS=, read -r ID CAT MODELO BATERIA DISP; do
```

En la línea 16 del script don_drones_manager.sh, el comando “tail -n +2” lee el archivo desde la segunda línea, salteando los encabezados. El pipe (|) envía esas filas al while.

Por defecto, Bash ejecuta ese while del pipe en una “subshell”, un entorno separado con sus propias copias de las variables.

Por eso:

1. El CONTADOR empieza en 0.
2. Dentro del while, su copia aumenta correctamente.
3. Cuando termina el bucle, esa copia se pierde.
4. El echo de la línea 21 muestra el contador original, que sigue siendo 0.

En el archivo flota.csv, hay 7 drones con batería menor a 20%:

```
SCO-002,Scout,FalconEye Mini,15,Si
CAR-003,Cargo,SkyHauler X1,8,Si
HEA-002,Heavy,TitanLift 20,19,Si
COO-002,Cooler,FreshWing R1,12,Si
RAC-002,Racer,BoltStrike V3,5,Si
HEA-004,Heavy,TitanLift Pro,3,Si
RAC-004,Racer,BoltStrike V3,18,Si
```

Para arreglarlo, tenemos que alimentar el bucle sin ponerlo después del pipe:

```
15          CONTADOR=0
16          while IFS=, read -r ID CAT MODELO BATERIA DISP; do
17              if [ "$BATERIA" -lt "20" ]; then
18                  CONTADOR=$((CONTADOR + 1))
19              fi
20          done < <(tail -n +2 "$ARCHIVO_FLOTA" )
21
22          echo "Drones con bateria critica: $CONTADOR"
23          ;;
```

Así, el while modifica el contador del mismo entorno donde luego se imprime.

Respuesta al problema 2

```
34          if [ "$BATERIA_ACTUAL" > "20" ]; then
```

En la línea 34 del script don_drones_manager.sh, el “>” no realiza una comparación numérica. Bash lo interpreta como una redirección de salida hacia un archivo llamado “20”, que puede crear o vaciar.

Una vez procesada la redirección, con una batería de 5, la condición equivale a: [“5”].

Eso comprueba si el texto “5” es no vacío. Como lo es, da verdadero y permite asignar el dron.

Para arreglarlo, cambiamos el “>” por el operador “-gt” que sirve para comparar y significa “mayor que”. Nos queda:

```
if [ "$BATERIA_ACTUAL" -gt 20 ]; then
```